SigmaStar Display
应用开发参考手册

REVISION HISTORY

Revision No.	Description	Date
0.1	• Initial release	04/06/2019

TABLE OF CONTENTS

1. SDK 说明

1.1. Project 目录说明

```
beal.wu@sigmastar:~/master_i6/project$ ls
board  configs  image  kbuild  makefile  release  setup_config.sh  tools
```

board	存放了相关配置信息，vdec 固件，mmap.ini 等，比如需要修改 framebuffer 输出格式的时候就需要修改/project/board/i2m/SSC011A-S01A/config/fbdev.ini 中的 FB_HWWIN_FORMAT(5=ARGB8888，6=ARGB1555) 缺省 fb 会被驱动为对应的格式
configs	存放编译 sdk，生成镜像的 config，如编译的第一步 ./setup_config.sh configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64
image	编译打包的主要目录，包含已经编译的 busybox 镜像以及源码，以及最终编译生成的烧录脚本以及烧录镜像。
kbuild	编译 sdk 需要用到的 kernel config 等设定
release	SDK 提供的头文件、lib、ko，其中包含各个模块的头文件以及数据结构
tools	一些 debug 工具

1.2. AppDemo 目录介绍

获取到 demo 的源码 code，从 github 下载

```
beal.wu@sigmastar:~/master_i6/sdk/verify/application/smarttalk$ ls
3partyLib  demoApp  image.mk  README.txt  RunFile
```

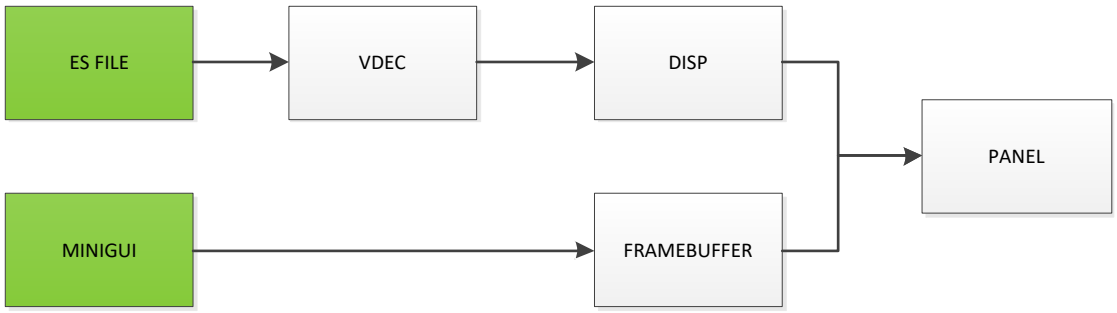
3partyLib	第三方库以及头文件，部分 lib 没有暂时没使用到，主要使用到了 minigui、xml、Cspotter 等
demoApp	测试程序的主要代码逻辑代码目录，实现了 gui 和 media 封装
image.mk	在编译 sdk 时，制作 image 过程中，拷贝 demo 运行需要的配置文件以及媒体素材
README.txt	手动执行程序的步骤，但是 spinor 的/etc 无法写，所以需要在制作 image 的时候 copy 进去
RunFile	运行 app 需要的配置文件以及媒体素材

1.2.1 如何编译 Demo

cd demoApp
./build_sample.sh (会根据 CMakeList.txt 中的规则编译 demo)
成功编译，会在 build 目录生成 mginit 的 bin 程序

1.2.2 DemoApp 实现的部分

Demo 主要实现了 UI 以及解码显示的功能模块展示，从 demo 中可以看到这样的



1.3. SSD201 常用模块说明

模块	功能描述
SYS	SYS 模块是整个 SDK 的内存管理，流传输控制，帧率设定等等
VDEC	解码，支持 H264、H265 解码，最大支持 1080P 1、 需要设置 ES 流的宽高 2、 需要设置解码输出 YUV 的宽高，输出的宽高必须和 DISP 的 input 宽高设定一样，VDEC 支持将 ES 流的宽高缩小，但是不支持放大
DIVP	YUV 图像处理模块，支持缩放以及格式转换，常用于同时绑定 DISP 和 VENC
DISP	图像显示模块，需要设置 Input port 属性，宽高需要注意和前一级绑定的模块输出宽高一致
VENC	编码支持 JPEG
PANEL	显示屏相关的配置，参考 demo 填写对应的屏参
GFX	2D 引擎，支持 Fill、Blit、Blending 等
FB	标准的 framebuffer 接口，设备节点为/dev/fb0，增加了一些扩展 ioctl，见 mstarFb.h

下面依次介绍一下各个常用模块的主要 API 使用，可以参考 Demo 来封装实现客户自己的媒体架构

1.3.1 SYS 模块

MI_S32 MI_SYS_Init(void); //使用模块 API 前需要 call 此 API
MI_S32 MI_SYS_Exit (void); //对应 Init

SDK 中各个模块如果需要传递 YUV Stream，那么就需要将对应的 Port 口绑定起来，这样上一级模块处理好的 YUV 数据就会送给下一级模块。

注：有些模块仅有 Input Port，有些模块仅有 Output Port，而有些则有 Input/Output Port，如上

1.3.2 VDEC 的 API 使用

```
MI_S32 ST_CreateVdecChannel(MI_S32 s32VdecChn, MI_S32 s32CodecType,
    MI_U32 u32Width, MI_U32 u32Height, MI_U32 u32OutWidth, MI_U32 u32OutHeight)
{
    MI_VDEC_ChnAttr_t stVdecChnAttr;
    MI_VDEC_OutputPortAttr_t stOutputPortAttr;

    memset(&stVdecChnAttr, 0, sizeof(MI_VDEC_ChnAttr_t));
    stVdecChnAttr.stVdecVideoAttr.u32RefFrameNum = 2;
    stVdecChnAttr.eVideoMode    = E_MI_VDEC_VIDEO_MODE_FRAME;
    stVdecChnAttr.u32BufSize    = 1 * 1024 * 1024;
    stVdecChnAttr.u32PicWidth   = u32Width;
    stVdecChnAttr.u32PicHeight  = u32Height;          ES流宽高
    stVdecChnAttr.u32Priority   = 0;
    stVdecChnAttr.eCodecType    = s32CodecType;       ES流格式
    stVdecChnAttr.eInplaceMode = 0;

    STCHECKRESULT(MI_VDEC_CreateChn(s32VdecChn, &stVdecChnAttr));
    STCHECKRESULT(MI_VDEC_StartChn(s32VdecChn));
    if (u32OutWidth > u32Width)
    {
        u32OutWidth = u32Width;
    }
    if (u32OutHeight > u32Height)
    {
        u32OutHeight = u32Height;
    }
    stOutputPortAttr.u16Width = u32OutWidth;
    stOutputPortAttr.u16Height = u32OutHeight;        输出宽高，需小于等于ES流宽高
    MI_VDEC_SetOutputPortAttr(s32VdecChn, &stOutputPortAttr);

    return MI_SUCCESS;
}«end ST_CreateVdecChannel»
```

创建好解码通道后，需要调用 MI_VDEC_SendStream 来将网络流或者录像文件送给 VDEC

```
if (MI_SUCCESS != (s32Ret = MI_VDEC_SendStream(vdecChn, &stVdecStream, s32TimeOutMs)))
{
    //ST_ERR("MI_VDEC_SendStream fail, chn:%d, 0x%X\n", vdecChn, s32Ret);
    fseek(readfp, -u32FpBackLen, SEEK_CUR);
}
```

销毁解码通道：

```
MI_S32 ST_DestroyVdecChannel(MI_S32 s32VdecChn)
{
    MI_S32 s32Ret = MI_SUCCESS;

    s32Ret = MI_VDEC_StopChn(s32VdecChn);      先stop，再destroy
    if (MI_SUCCESS != s32Ret)
    {
        printf("%s %d, MI_VENC_StopRecvPic %d error, %X\n", __func__, __LINE__, s32VdecChn, s32Ret);
    }
    s32Ret |= MI_VDEC_DestroyChn(s32VdecChn);
    if (MI_SUCCESS != s32Ret)
    {
        printf("%s %d, MI_VENC_StopRecvPic %d error, %X\n", __func__, __LINE__, s32VdecChn, s32Ret);
    }

    return s32Ret;
}
```

1.3.3 DISP

设置 disp dev 的属性，并且使能 dev

```
stPubAttr.stSyncInfo.u16Vact = stPanelParam.u16Height;
stPubAttr.stSyncInfo.u16Vbb = stPanelParam.u16VSyncBackPorch;
stPubAttr.stSyncInfo.u16Vfb = stPanelParam.u16VTotal - (stPanelParam.u16VSyncWidth + stPanelParam.u16Height +
stPubAttr.stSyncInfo.u16Hact = stPanelParam.u16Width;
stPubAttr.stSyncInfo.u16Hbb = stPanelParam.u16HSyncBackPorch;
stPubAttr.stSyncInfo.u16Hfb = stPanelParam.u16HTotal - (stPanelParam.u16HSyncWidth + stPanelParam.u16Width +
stPubAttr.stSyncInfo.u16Bvact = 0;
stPubAttr.stSyncInfo.u16Bvbb = 0;
stPubAttr.stSyncInfo.u16Bvfb = 0;
stPubAttr.stSyncInfo.u16Hpw = stPanelParam.u16HSyncWidth;
stPubAttr.stSyncInfo.u16Vpw = stPanelParam.u16VSyncWidth;
stPubAttr.stSyncInfo.u32FrameRate = stPanelParam.u16DCLK*1000000/(stPanelParam.u16HTotal*stPanelParam.u16VTot
stPubAttr.eIntfSync = E_MI_DISP_OUTPUT_USER;
stPubAttr.eIntfType = E_MI_DISP_INTF_LCD;
stPubAttr.u32BgColor = YUYV_BLACK;
MI_DISP_SetPubAttr(dispDev, &stPubAttr);
MI_DISP_Enable(dispDev);
```

屏参配置

使能disp device

绑定 layer 和 disp dev

```
MI_DISP_BindVideoLayer(dispLay, dispDev);
MI_DISP_EnableVideoLayer(dispLay);
```

绑定layer

设置 disp input port 的参数以及使能

```
memset(&stInputPortAttr, 0, sizeof(stInputPortAttr));

u32InputPort = pstDispChnInfo->stInputPortAttr[i].u32Port;
ExecFunc(MI_DISP_GetInputPortAttr(DispLayer, u32InputPort, &stInputPortAttr), MI_SUCCESS);
stInputPortAttr.stDispWin.u16X      = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16X;
stInputPortAttr.stDispWin.u16Y      = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16Y;
stInputPortAttr.stDispWin.u16Width  = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16Width;
stInputPortAttr.stDispWin.u16Height = pstDispChnInfo->stInputPortAttr[i].stAttr.stDispWin.u16Height;
stInputPortAttr.u16SrcWidth  = pstDispChnInfo->stInputPortAttr[i].stAttr.u16SrcWidth;
stInputPortAttr.u16SrcHeight = pstDispChnInfo->stInputPortAttr[i].stAttr.u16SrcHeight;

ExecFunc(MI_DISP_SetInputPortAttr(DispLayer, u32InputPort, &stInputPortAttr), MI_SUCCESS);
ExecFunc(MI_DISP_GetInputPortAttr(DispLayer, u32InputPort, &stInputPortAttr), MI_SUCCESS);
ExecFunc(MI_DISP_EnableInputPort(DispLayer, u32InputPort), MI_SUCCESS);
ExecFunc(MI_DISP_SetInputPortSyncMode(DispLayer, u32InputPort, E_MI_DISP_SYNC_MODE_FREE_RUN), MI_SUCCESS);
```

设定DISP port的参数，可以理解为视频区域的设定，disp input port == disp video window

需和前级输出YUV宽高一致。

1.3.4 Panel

Panel 部分，ttl 的 lcd 仅需要根据 spec 填写屏参，然后执行如下 api 即可，屏参参考 demo 的 LCD 头文件

```
MI_PANEL_Init(E_MI_PNL_LINK_TTL);
MI_PANEL_SetPanelParam(&stPanelParam);
```

Panel初始化设定

1.3.5 Framebuffer

初始化 FB

```
g_fbFd = open (devfile, O_RDWR);
if (g_fbFd == -1)
{
    perror("Error: cannot open framebuffer device");
    exit(0);
}
//get fb_fix_screeninfo
if (ioctl(g_fbFd, FBIOGET_FSCREENINFO, &finfo) == -1)
{
    perror("Error reading fixed information");
    exit(0);
}
//get fb_var_screeninfo
if (ioctl(g_fbFd, FBIOGET_VSCREENINFO, &vinfo) == -1)
{
    perror("Error reading variable information");
    exit(0);
}
/* Figure out the size of the screen in bytes */
g_screensize = finfo.smem_len;

/* Map the device to memory */
frameBuffer =
    (char *) mmap(0, g_screensize, PROT_READ | PROT_WRITE, MAP_SHARED, g_fbFd, 0);
if (frameBuffer == MAP_FAILED)
{
    perror("Error: Failed to map framebuffer device to memory");
    exit(0);
}
// clear framebuffer
drawRect(0, 0, vinfo.xres, vinfo.yres, ARGB888_BLUE);
```

Framebuffer是标准的fb接口，如果是自行对接UI，那么需要如左侧初始化Fb，然后直接对frameBuffer这个地址读写即可。
Fb会跟据/config/fbdev.ini中的配置初始化设备

FB 也扩展了一些 iotcl，如 colorkey 和 Hide/Show。

Show/Hide:

```
if (ioctl(g_fbFd, FBIOGET_SHOW,&bShown)<0) {
    perror("Error: failed to FBIOGET_SHOW");
    exit(0);
}
```

ColorKey:

```
MI_FB_ColorKey_t colorKeyInfo;
if (ioctl(g_fbFd, FBIOGET_COLORKEY,&colorKeyInfo) < 0) {
    ST_ERR("Error: failed to FBIOGET_COLORKEY\n");
    exit(0);
}

colorKeyInfo.bKeyEnable = TRUE;
colorKeyInfo.u8Red = PIXEL8888RED(u32ColorKeyVal);
colorKeyInfo.u8Green = PIXEL8888GREEN(u32ColorKeyVal);
colorKeyInfo.u8Blue = PIXEL8888BLUE(u32ColorKeyVal);

//convertColorKeyByFmt(&colorKeyInfo);
if (ioctl(g_fbFd, FBIOSET_COLORKEY, &colorKeyInfo) < 0) {
    ST_ERR("Error: failed to FBIOGET_COLORKEY");
    exit(0);
}
```

扩展 ioctl：

```
#define FBIOGET_SCREEN_LOCATION _IOR(FB_IOC_MAGIC, 0x60, MI_FB_Rectangle_t)
#define FBIOSET_SCREEN_LOCATION _IOW(FB_IOC_MAGIC, 0x61, MI_FB_Rectangle_t)

#define FBIOGET_SHOW _IOR(FB_IOC_MAGIC, 0x62, MI_BOOL)
#define FBIOSET_SHOW _IOW(FB_IOC_MAGIC, 0x63, MI_BOOL)

#define FBIOGET_GLOBAL_ALPHA _IOR(FB_IOC_MAGIC, 0x64, MI_FB_GlobalAlpha_t)
#define FBIOSET_GLOBAL_ALPHA _IOW(FB_IOC_MAGIC, 0x65, MI_FB_GlobalAlpha_t)

#define FBIOGET_COLORKEY _IOR(FB_IOC_MAGIC, 0x66, MI_FB_ColorKey_t)
#define FBIOSET_COLORKEY _IOW(FB_IOC_MAGIC, 0x67, MI_FB_ColorKey_t)

#define FBIOGET_DISPLAYLAYER_ATTRIBUTES _IOR(FB_IOC_MAGIC, 0x68, MI_FB_DisplayLayerAttr_t)
#define FBIOSET_DISPLAYLAYER_ATTRIBUTES _IOW(FB_IOC_MAGIC, 0x69, MI_FB_DisplayLayerAttr_t)

#define FBIOGET_CURSOR_ATTRIBUTE _IOR(FB_IOC_MAGIC, 0x70, MI_FB_CursorAttr_t)
#define FBIOSET_CURSOR_ATTRIBUTE _IOW(FB_IOC_MAGIC, 0x71, MI_FB_CursorAttr_t)
```